PHASE 1 MQTT / REFERENCE & VALIDATION / 01

Phase 1 MQTT integration

Smart Parking System
Runtime Audit & Continuation Report

Three ESP32 boards | Twelve HC-SR04 slots | Local Mosquitto

Prepared 10 November 2026. Times are explicitly labelled UTC or IST (Asia/Calcutta). This report records measured results, operator statements and implementation details separately.

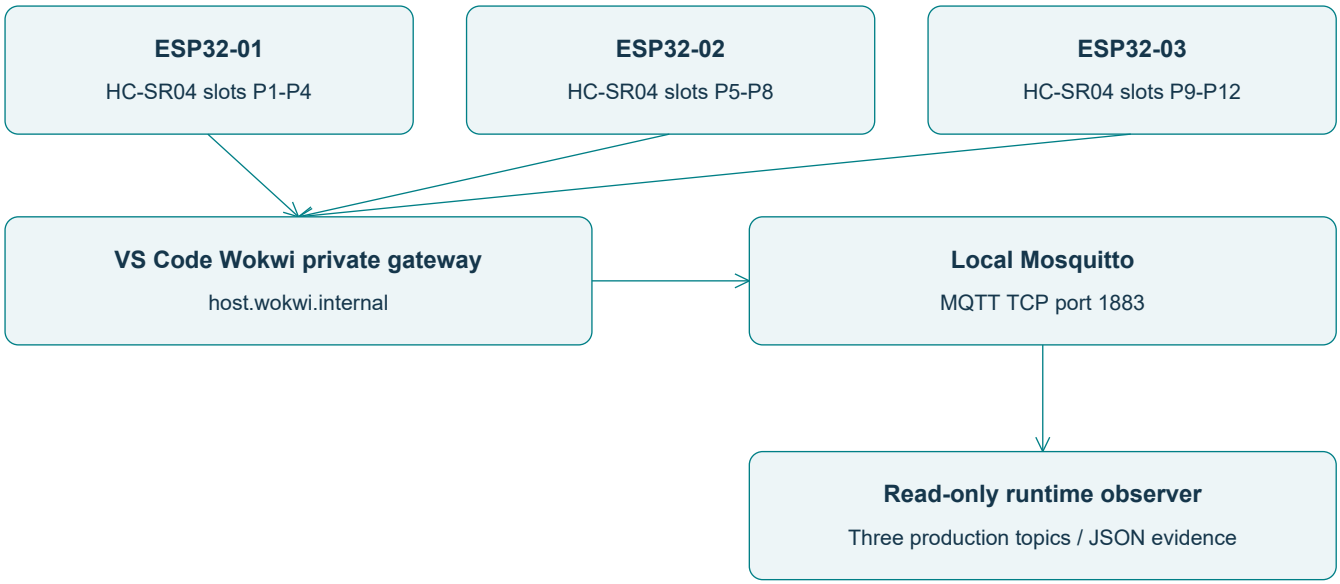
DELIVERY OPERATOR-OBSERVED; DETAILED TESTS OPEN. All three firmware builds and local Python broker checks passed. The operator observed all three simulations delivering telemetry to their production topics with assigned slots P1-P12. No original screenshot or payload archive is available. Detailed cadence, timestamps, state transitions and recovery remain unverified; no complete acceptance-suite pass is claimed.

Item	Recorded basis
Initial source baseline	e0ae26d729341779d06df91257c2735ac09707b4
Production captures	3 separate observation windows; esp32_01: 0 valid messages, esp32_02: 0 valid messages, esp32_03: 0 valid messages
Operator statements	Later operator report confirms simultaneous simulations and subscriber delivery on all three production topics with P1-P12. Recorded as operator-observed only; original subscriber output and exact per-board payloads are unavailable.
CLI access	Wokwi CLI installed; token not available in process, user or machine environment at initial inspection. Operator selected the VS Code launch route.
Scope	ESP32 MQTT runtime audit, diagnostic tools and documentation only. No FastAPI, SQLite, ML or dashboard implementation.

The original four-slot code remains preserved at tag backup/4-slot-serial-baseline. The 12-slot source and board workspaces are preserved in Git. A documentation commit does not change the acceptance status of the runtime tests.

PHASE 1 MQTT / REFERENCE & VALIDATION / 02

Architecture and boundaries



Three separate Wokwi simulations; the combined diagram is an overview.

End-to-end path to prove

For each board: sensor measurement -> unchanged occupancy classification -> a completed four-slot snapshot -> MQTT JSON publish -> broker forwarding -> a fresh message on the matching production subscription. A Serial connection statement alone does not prove the final delivery step.

Wokwi does not support multiple microcontrollers executing in one project. Each board therefore has its own simulation folder and VS Code workspace. The combined esp32/diagram.json is a wiring overview, not a three-controller runtime configuration. [1]

Included and excluded components

Included: Wi-Fi, private-gateway routing, board identities, MQTT reconnect logic, JSON snapshots, Serial diagnostics and evidence capture. Excluded: database ingestion, occupancy aggregation across devices, REST APIs, prediction, dashboard wiring and deployment. The local broker is shared; firmware handles only each board's four assigned sensors.

PHASE 1 MQTT / REFERENCE & VALIDATION / 03

Circuit mapping and detection behavior

Board	Slot	TRIG GPIO	ECHO GPIO after divider
ESP32-01	P1	13	34
ESP32-01	P2	14	35
ESP32-01	P3	25	32
ESP32-01	P4	26	33
ESP32-02	P5	13	34
ESP32-02	P6	14	35
ESP32-02	P7	25	32
ESP32-02	P8	26	33
ESP32-03	P9	13	34
ESP32-03	P10	14	35
ESP32-03	P11	25	32
ESP32-03	P12	26	33

Every sensor VCC connects to its assigned board's 5V, and GND connects to that board's GND.1. Every ECHO has its own 1,000-ohm series resistor and 1,800-ohm shunt resistor to ground; the ESP32 input is at the divider midpoint. No signal node is shared between slots or boards.



Calculated midpoint for a 5 V ECHO high: 5 x 1800 / (1000 + 1800) = 3.214 V. This is a calculation, not a voltage measured in Wokwi. The simulator's analog support is limited; physical hardware still requires appropriate components, supply checks and measured logic levels. [4]

Detection remains: 10 us trigger; pulseIn HIGH with 30 ms timeout; distance_cm = echo_us / 58; valid range 2-400 cm; OCCUPIED at or below the configurable 20 cm threshold; VACANT above it; UNKNOWN on timeout or out-of-range. Invalid distance is null in MQTT. Measurements are sequential, with a 60 ms quiet interval. [5]

PHASE 1 MQTT / REFERENCE & VALIDATION / 04

Firmware implementation and recovery design

Board-specific compilation

platformio.ini defines esp32_01, esp32_02 and esp32_03 with PARKING_BOARD_ID=1, 2 and 3. A missing or invalid ID fails compilation. The selected branch contains exactly four slot entries. MQTT uses the pinned 256dpi/MQTT@2.5.3 library; the Espressif platform remains pinned at 8.0.0. No Python environment dependencies were changed for firmware integration.

Publish call path

setup initializes Serial, TRIG outputs and ECHO inputs, starts station-mode Wi-Fi and configures the MQTT transport. Each loop services the network, measures and prints the four local slots, services the network during quiet intervals, then calls publishTelemetry(samples). That call happens after every completed scan. A call is not necessarily a publish: disconnected or not-yet-due conditions return early.

An eligible publication builds one bounded JSON snapshot and calls gMqtt.publish(topic, payload, false, 1). The false argument disables retention; 1 requests QoS 1. A true result triggers the success log and sequence increment. False is logged as failure and does not prove broker delivery. PARKING_MQTT_LOG_PAYLOAD defaults to 1. Serial prints the connected/due gate each scan, the production topic before an attempt, and the exact JSON only after publish returns true. last_error and return_code diagnose failure without printing credentials. Set the flag to 0 after diagnostics if desired.

Timing and disconnection behavior

Scans target one second and publications target five seconds. Network service runs between sensors and in short sleep slices. There is no infinite wait-until-connected loop. TCP and MQTT acknowledgement timeouts are configured at about one second, while DNS or stack delays can still extend a scan. Retry delays are capped at 30 seconds; a successful reconnect resets publication gating so a new scan can be sent.

No disconnected history is queued. A failed publication is rate-limited, and the next eligible attempt uses a fresh scan. QoS 1 can duplicate messages. Sequence counts acknowledged sends and resets on reboot; it is not a globally unique ingestion key. A future backend will need explicit restart/session identity and duplicate/freshness policy.

Inspection versus execution: the source contains the post-scan publish call. The call path is source-verified. Subscriber delivery is operator-observed. No exact Serial log is archived, and which binary produced that observation is not independently established.

PHASE 1 MQTT / REFERENCE & VALIDATION / 05

MQTT configuration and workspaces

Build / device	Client ID	Production topic
esp32_01	smartparking-esp32_01	smartparking/esp32_01/telemetry
esp32_02	smartparking-esp32_02	smartparking/esp32_02/telemetry
esp32_03	smartparking-esp32_03	smartparking/esp32_03/telemetry

Setting	Value / interpretation
ESP32 broker	host.wokwi.internal:1883 through the private gateway; not localhost.
PC observer broker	127.0.0.1:1883. The observer runs on the PC, outside the simulated ESP32.
Wi-Fi	Wokwi-GUEST, empty password, channel 6.
Payload cadence	Five seconds when connected; exactly four local slots per snapshot.
Delivery	MQTT 3.1.1, QoS 1, retain=false. Distinct client IDs prevent one board replacing another.
Private overrides	Optional gitignored esp32/include/config.h; public defaults in mqtt_config.h. Credentials are never printed or included in evidence.

VS Code Wokwi includes a private gateway; no inbound port-forward rule is needed for outbound MQTT. A free browser public gateway cannot reach the PC's local broker. The expected hostname and compiled paths were checked, and delivery through the simulation path was subsequently operator-observed; no independently archived runtime trace is available. [2,3]

Open smart-parking-esp32-01.code-workspace, -02 or -03 in separate VS Code windows. Each puts its local simulations/esp32-0N folder first, then the source repository. Its wokwi.toml uses ../../.pio/build/esp32_0N/firmware.bin and the matching ELF. Use Wokwi: Start Simulator or Play on the local one-board diagram, never on the three-board overview.

The existing Windows Mosquitto service configuration was not changed. Read-only observation identified Mosquitto 2.1.2. The service config hash is retained in the evidence metadata for comparison. A broker-side subscriber receiving synthetic test traffic does not establish private-gateway reachability.

PHASE 1 MQTT / REFERENCE & VALIDATION / 06

Telemetry schema and example

Field	Meaning
schema_version / device_id	Version 1; esp32_01, esp32_02 or esp32_03.
sequence / uptime_ms	Acknowledged-send counter and device millis at payload creation; both can restart/wrap.
timestamp / clock_synced	Root publication time in UTC and valid-clock indicator. Null/false before wall time is usable.
slots[]	Exactly four uniquely assigned slot objects, containing slot_id, distance_cm, occupancy_status and measurement timestamp.
distance_cm / occupancy_status	Finite valid range value with OCCUPIED/VACANT, or null with UNKNOWN.

Illustrative only: no production payload was captured in the initial observation windows.

```
{
  "schema_version": 1,
  "device_id": "esp32_01",
  "sequence": 0,
  "timestamp": null,
  "clock_synced": false,
  "uptime_ms": 1000,
  "slots": [
    {"slot_id":"P1","distance_cm":10.0,
     "occupancy_status":"OCCUPIED","timestamp":null},
    {"slot_id":"P2","distance_cm":100.0,
     "occupancy_status":"VACANT","timestamp":null},
    {"slot_id":"P3","distance_cm":null,
     "occupancy_status":"UNKNOWN","timestamp":null},
    {"slot_id":"P4","distance_cm":100.0,
     "occupancy_status":"VACANT","timestamp":null}
  ]
}
```

After asynchronous SNTP establishes usable wall time, timestamps use ISO 8601 UTC, for example 2026-11-10T12:00:00Z. The initial snapshot can have null or mixed slot timestamps if synchronization occurs during a scan. Do not treat uptime as Unix time, or claim NTP success merely because the field exists.

PHASE 1 MQTT / REFERENCE & VALIDATION / 07

Executed testing procedure and commands

The operator selected the VS Code route because no CLI token was configured. The agent did not control the browser or IDE. The operator reported three running simulations and later supplied short Wi-Fi/MQTT connection statements; exact per-board publication output was subsequently requested. The operator later explicitly reported subscriber delivery on all three topics with P1-P12. That statement is archived separately; no Serial lines or production payloads have been invented.

Read-only production captures

```
.\.venv\Scripts\python.exe esp32/tools/verify_mqtt_runtime.py `
  --duration 90 --label baseline `
  --output reports/evidence/phase1_mqtt_baseline.json

.\.venv\Scripts\python.exe esp32/tools/verify_mqtt_runtime.py `
  --duration 60 --label operator_confirmed_running `
  --output reports/evidence/phase1_mqtt_running.json

.\.venv\Scripts\python.exe esp32/tools/verify_mqtt_runtime.py `
  --duration 45 --label operator_reported_mqtt_connected `
  --output reports/evidence/phase1_mqtt_connected.json
```

The first two captures also supplied explicit expected VACANT states for P1-P12, matching the configured 100 cm starting distances. The observer subscribes to exactly the three production topics at QoS 1, ignores retained packets and exact duplicate identities, validates the documented JSON fields, and saves only whitelisted telemetry fields. It never publishes to production topics or overwrites an evidence file.

Broker isolation and health checks

```
.\.venv\Scripts\python.exe esp32/tools/check_mqtt.py --smoke --timeout 5
```

This passed using three Python publishers and one subscriber on a unique smartparking/validation/phase1/... namespace. Fixtures were explicitly labelled local_python_probe_not_esp32. A separate read-only $SYS subscription checked broker version, uptime and connected-client count while the production observer was open. No service restart or configuration edit was made.

Diagnostic firmware builds: esp32_01, esp32_02 and esp32_03 all SUCCESS; each uses 48,636 bytes RAM and 997,633 bytes flash. All three diagnostic host test programs compiled with --compile-only; they were not executed because of prior Windows Application Control blocks. Earlier circuit/config checks and an initial ESP32-01 host run passed. These results do not substitute for simulation execution. The interrupted later diagnostic-restart capture produced no saved record and supplies no test result.

PHASE 1 MQTT / REFERENCE & VALIDATION / 08

Actual production-topic observations

Capture label	Start (10 Nov)	Requested window	Messages: 01 / 02 / 03
baseline	19:51:37 IST	90.0 s	0 / 0 / 0
connected	19:59:39 IST	45.0 s	0 / 0 / 0
running	19:53:49 IST	60.0 s	0 / 0 / 0

These are separate windows, not one continuous run. Every capture records its actual elapsed wall time and UTC start/end times in the JSON evidence. A zero count means no fresh valid production payload was observed in that window; it does not establish the precise cause.

Check	Result	Interpretation
Observer -> broker CONNACK	PASSED	Successful connection acknowledgements were recorded.
Three production SUBACKs	PASSED	Broker granted QoS 1 subscriptions for all three topics.
Python broker pub/sub	PASSED	Three synthetic publishers worked in an isolated validation namespace.
Three-topic delivery / P1-P12	OPERATOR OBSERVED	Operator saw all three production topics while simulations ran simultaneously; no original payload archive.
Archived production payloads	NOT CAPTURED	esp32_01: 0, esp32_02: 0, esp32_03: 0
Periodic firmware delivery	UNVERIFIED	No production intervals can be computed without messages.
Actual timestamps / NTP	UNVERIFIED	No production timestamps captured in the initial windows.
Occupancy transitions	UNVERIFIED	No received state changes correlated with operator distance changes.
MQTT reconnect / recovery	NOT RUN	No controlled outage and recovery evidence was supplied.

Acceptance remains open. Delivery is operator-observed, while the saved earlier windows contain no messages. Neither source proves the remaining detailed tests passed. These evidence classes are intentionally preserved separately.

PHASE 1 MQTT / REFERENCE & VALIDATION / 09

Broker diagnostics and later observation

System topic	Observed value	Context
$SYS/broker/version	mosquitto version 2.1.2	Read-only broker diagnostic; retained flag: True
$SYS/broker/clients/connected	2	Read-only broker diagnostic; retained flag: False
$SYS/broker/messages/received	590	Read-only broker diagnostic; retained flag: False
$SYS/broker/uptime	17193 seconds	Read-only broker diagnostic; retained flag: False

The broker reported two connected clients while two diagnostic observers were open. That count is consistent with the diagnostic subscribers alone; it does not support three additional connected ESP32 clients at that observed moment. Broker statistics update periodically, so this is supporting evidence rather than an exact per-client trace.

What can and cannot be concluded

Confirmed: the local broker responds, accepts the expected subscriptions and forwards isolated test messages. The later operator observation establishes reported subscriber delivery, but not an independently archived broker route, loaded binary identity, exact JSON fields or the reason earlier observation windows were empty.

Possible causes include a stale simulation image, a topic or broker mismatch, a gateway route problem, or a connected client that is not reaching its publish path. The later operator observation does not retrospectively diagnose the earlier empty windows. These are hypotheses. No circuit rewrite, broker reconfiguration or invented payload is justified by a zero-message observation alone.

Serial evidence required next

For every board, capture the startup banner, assigned slot lines, Wi-Fi status, exact MQTT-connected topic, publish-check gate, publish-attempt result and acknowledged JSON payload. An unedited copied log is preferred. Board names and slot IDs should agree with the selected workspace and build target. Credentials must not be included.

Timestamp, state and reconnect acceptance

Steady-state telemetry

Run all three independent simulations and observe for at least 60-90 seconds after connection. Require at least three fresh unique messages per device, exactly P1-P4 / P5-P8 / P9-P12, valid JSON, correct UNKNOWN/null rules and no foreign slots. Compare wall-clock arrival intervals with device uptime increments. Approximately five seconds is the target, but simulator speed can stretch wall time; report both measurements.

UTC and sensor states

Permit explicit null timestamps during startup. Then require multiple fully timestamped messages, UTC Z formatting, sensible ordering of measurement and publication times, and comparison with receiver UTC time. A non-null field alone is not proof that NTP or clock accuracy passed. Change P1, P5 and P9 individually from 100 cm to 10 cm and back, preserving the other slots. Correlate received transitions with operator changes. Test timeout/UNKNOWN only in a disposable fault-injection copy, not by damaging the verified circuit.

Controlled MQTT recovery, after baseline

One non-service method is a brief test-client connection using a selected ESP32's exact client ID. MQTT replacement disconnects that single ESP32 session; immediately disconnect the probe and observe the ESP32 reconnect and resume fresh production messages. Record the induced event time, Serial logs, sequence/uptime behavior, delivery gap and recovery time. Do not keep the duplicate client connected or cause a reconnect contest. This test has not been performed or operator-confirmed in this audit.

A simulator-side Wi-Fi interruption is a separate test and remains unverified. Do not treat a broker-client reconnect as proof of Wi-Fi recovery. Restarting the shared Windows Mosquitto service may affect other clients and is not necessary for the first targeted recovery test. Keep service settings unchanged unless a diagnosed routing/authentication need is explained and authorized.

Backend gate: Three-board delivery has operator confirmation; timestamp/state behavior and controlled recovery remain open acceptance items. This report does not authorize starting FastAPI or SQLite.

PHASE 1 MQTT / REFERENCE & VALIDATION / 11

Troubleshooting and continuation commands

Observation	Next check
No Wi-Fi connected line	Confirm latest image, Wokwi-GUEST, channel 6, simulator network and licensed private-gateway availability.
Wi-Fi connected; MQTT fails	Check host.wokwi.internal:1883 in the effective firmware, broker availability and authentication. Do not replace it with ESP32 localhost.
MQTT connected; no publish attempt	Look at publish-check connected/due gates and whether all four measurements complete.
Publish attempt fails	Capture library transport/return codes and broker-side symptoms. Do not label the attempted JSON delivered.
Acknowledged payload; observer sees none	Compare exact topic and broker endpoint, selected workspace, loaded firmware image and duplicate client IDs.
All slots UNKNOWN at 100 cm	Inspect ECHO pin paths and limited resistor simulation before claiming occupancy detection passed.
Clock remains unsynchronized	Check gateway DNS/UDP internet access to SNTP servers; retain null timestamps instead of fabricating dates.

Rebuild only if firmware/configuration changed

```
$env:PLATFORMIO_CORE_DIR = Join-Path (Get-Location).Path 'tmp\platformio-core'
.\tmp\platformio-venv\Scripts\python.exe -m platformio run --project-dir esp32
```

Capture a new window without replacing earlier results

```
.\.venv\Scripts\python.exe esp32/tools/verify_mqtt_runtime.py `
  --duration 90 --label after_serial_diagnosis `
  --output reports/evidence/phase1_mqtt_after_diagnosis.json
```

Use a new evidence filename for every test window. Keep the failed/empty captures: they explain why a later fix was needed and prevent retrospective replacement of earlier results with successful fixtures. The observer is read-only; no test tool should spoof ESP32 traffic on production topics to manufacture a pass.

PHASE 1 MQTT / REFERENCE & VALIDATION / 12

Evidence manifest and project references

Evidence file	Purpose
phase1_broker_system.json	read_only_mosquitto_system_topics
phase1_diagnostic_build.json	diagnostic_firmware_build_and_operator_clarification
phase1_environment.json	environment / operator / artifact record
phase1_mqtt_baseline.json	baseline
phase1_mqtt_connected.json	operator_reported_mqtt_connected
phase1_mqtt_running.json	operator_confirmed_running
phase1_operator_observation.json	operator_observed_runtime_verification

Firmware artifact record

esp32_01: 1,009,712 bytes; SHA-256: cd55ee105289ec26ec05441a71afb0c08769dc690d3897a97d254bc07a47c6a5

esp32_02: 1,009,712 bytes; SHA-256: 5bae0562976dd045c1d42ca68d8117021dc49416e83f5437b7f9af5f90b846d7

esp32_03: 1,009,712 bytes; SHA-256: 3554b32d7e074b60056d4c4a8a4a972adf4165c5ad9ddf44d4538d6d1cbd180e

Hashes identify the recorded binary files; they do not prove that a running simulator loaded those files. These are the diagnostic build hashes; the initial baseline hashes remain in phase1_environment.json. The operator observation was not tied to a verified binary hash. Do not embed API keys, passwords, personal IDE state or unrelated logs in committed reports.

Primary external references

[1] Wokwi FAQ: multiple microcontrollers: https://docs.wokwi.com/faq

[2] Wokwi VS Code project configuration: https://docs.wokwi.com/vscode/project-config

[3] ESP32 Wi-Fi and gateways: https://docs.wokwi.com/guides/esp32-wifi

[4] Wokwi resistor simulation limits: https://docs.wokwi.com/parts/wokwi-resistor

[5] HC-SR04 simulation and controls: https://docs.wokwi.com/parts/wokwi-hc-sr04

[6] Arduino MQTT library: https://github.com/256dpi/arduino-mqtt

[7] Wokwi CLI usage and token prerequisite: https://docs.wokwi.com/wokwi-ci/cli-usage

Repository continuation files: esp32/MQTT.md, esp32/README.md, the three board workspaces, esp32/tools/verify_mqtt_runtime.py and reports/evidence/. This PDF is generated by reports/generate_phase1_report.py using ReportLab, without modifying the project's Python dependency snapshot.